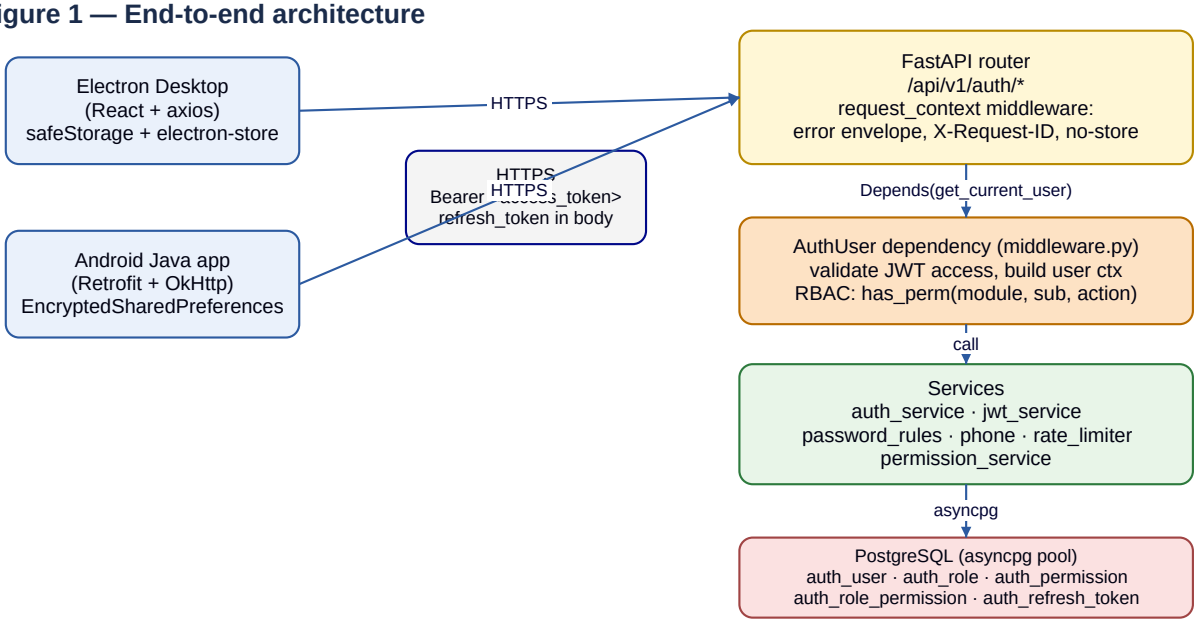
Authentication & Authorization

Detailed flow diagrams and frontend payload reference for the Candor Consumption FastAPI backend.

Base URL (prod): https://desktop-backend-vhf0.onrender.com
Base URL (local): http://localhost:8000
Auth prefix: /api/v1/auth
Header on every call after login: Authorization: Bearer <access_token>
Algorithm: JWT HS256 · Issuer: candor-consumption
Access TTL: 900 s (15 min) · **Refresh TTL:** 28800 s (8 h) — read from response, never hardcode.

1. End-to-end architecture

Figure 1 — End-to-end architecture



Two clients (Electron desktop, native Android) speak to the same FastAPI auth router. Every response carries an error-envelope-compatible shape, X-Request-ID, Cache-Control: no-store, and X-Content-Type-Options: nosniff. The get_current_user dependency validates the bearer token, then the service layer (jwt, password rules, rate limiter, permissions, phone normalizer) talks to Postgres via an asyncpg pool.

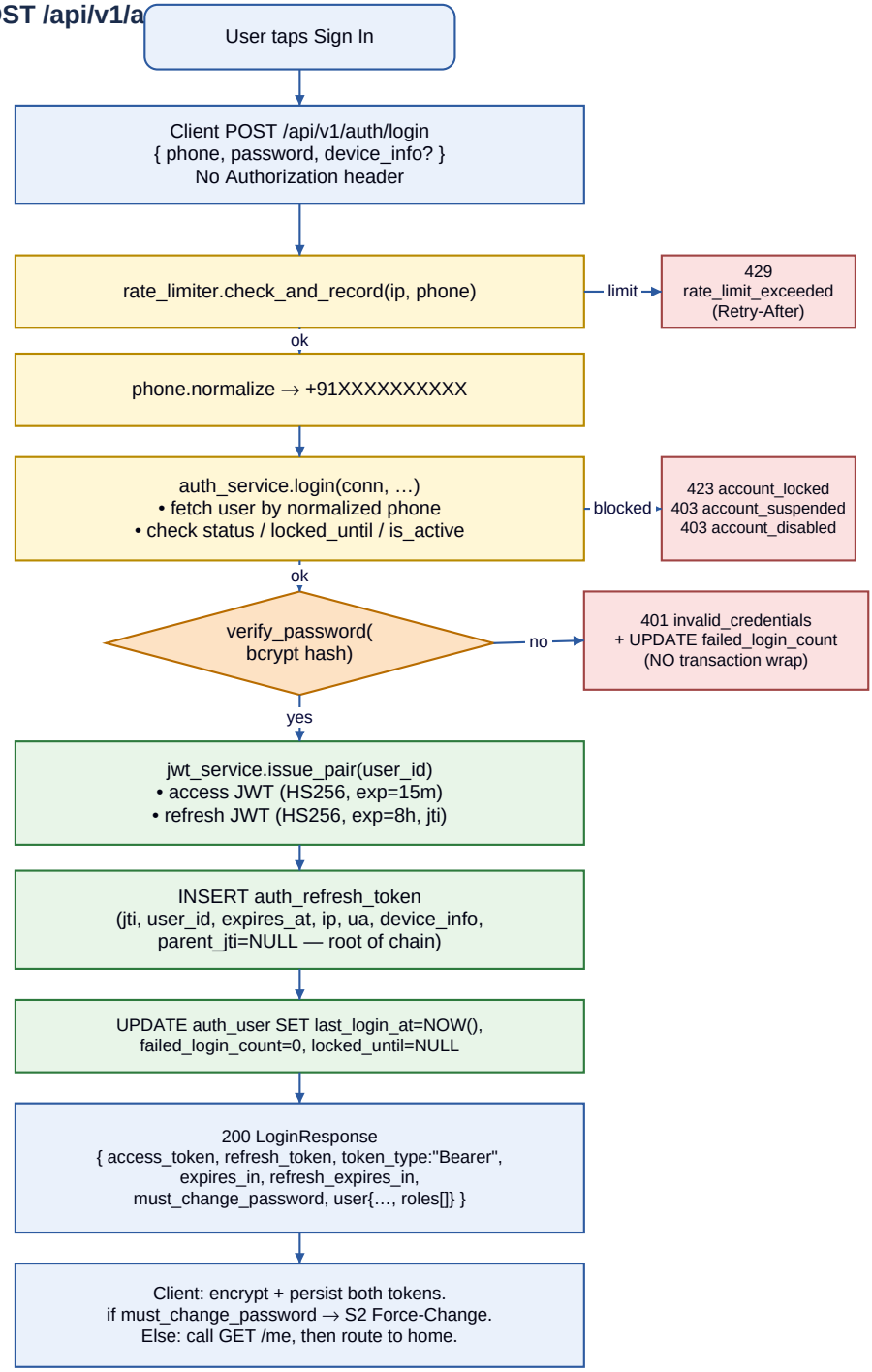
2. Hard rules (frontend MUST follow)

#	Rule (frontend MUST follow)
1	Never persist tokens in plain storage. Electron: safeStorage.encryptString; Android: EncryptedSharedPreferences + MasterKey AES256_GCM.
2	Send refresh_token ONLY on /auth/refresh and /auth/logout. Access token on every other call.
3	Never log tokens. Mask as 'Bearer ****' and strip from crash reports.
4	Treat JWT as opaque. May decode exp to schedule proactive refresh — never to make security decisions.
5	permissions[] from /me is a UX hint only. Server is authoritative on every call.
6	Capture X-Request-ID (or details.request_id) and surface it in every user-facing error toast.
7	Phone: send raw user input (4 accepted formats). Server normalizes to E.164.
8	/refresh rotates BOTH tokens. Replace both atomically; reusing an old refresh revokes the whole chain.
9	After /password/change, other devices are revoked. Show 'Other devices signed out (N)' toast.
10	423 account_locked and 429 rate_limit_exceeded are user-recoverable. Show timer; do NOT auto-retry.
11	Concurrent 401s → exactly ONE /refresh; queue others on the result (single-flight).
12	token_reuse_detected, invalid_refresh_token, account_disabled/suspended, /refresh 401 → clear + login.

3. Login flow — POST /api/v1/auth/login

Public endpoint. Rate-limited per (ip, phone). Failed-password UPDATE runs in autocommit (do **not** wrap in a transaction — that would roll the counter back and disable lockout).

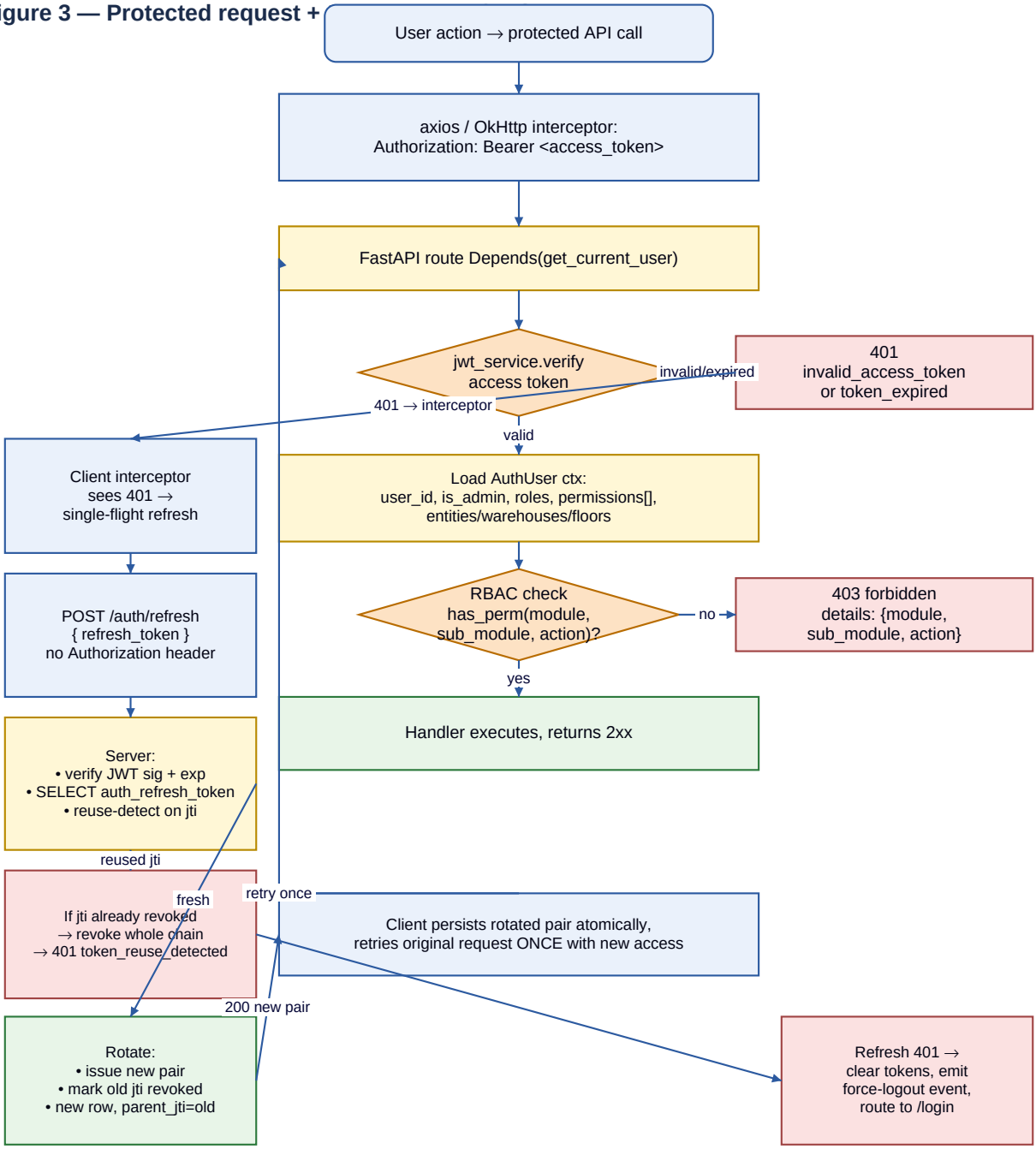
Figure 2 — Login flow (POST /api/v1/a



4. Protected request + transparent refresh

On every protected route, FastAPI's Depends(get_current_user) verifies the JWT and loads the AuthUser context. A 401 from a normal call triggers the client's single-flight refresh: exactly one /auth/refresh runs and queued requests wait on its result. The original request retries *once*; another 401 forces re-login. Refresh-token rotation is mandatory — reusing an old jti revokes the entire chain.

Figure 3 — Protected request +

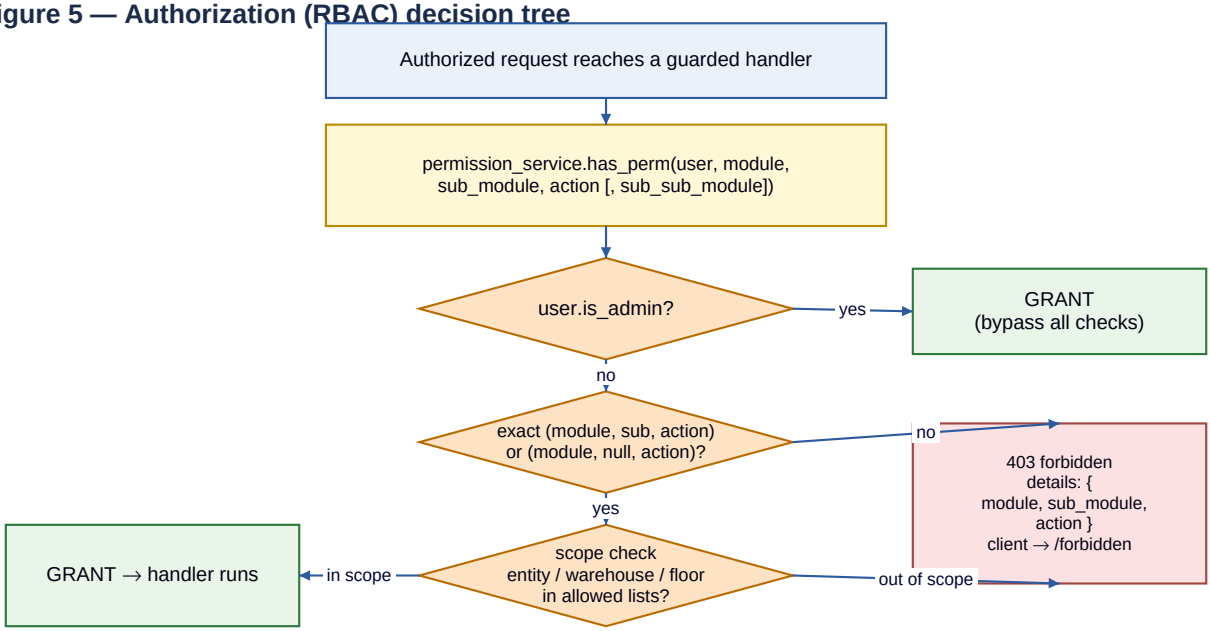


Source: modules/auth/router.py · modules/auth/schemas.py · Plan/Frontend_Authentication_and_Authorization.md

5. Authorization (RBAC) decision tree

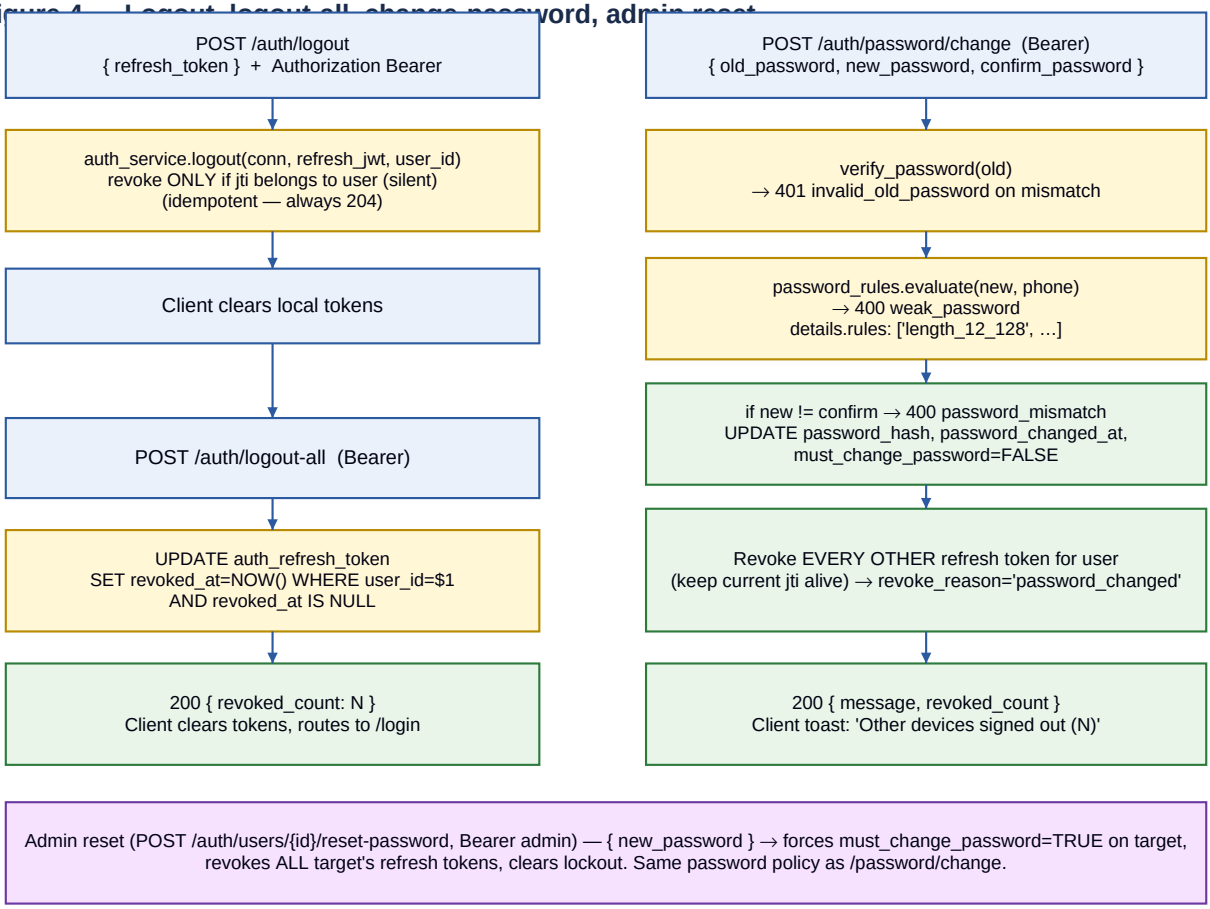
Admins (is_admin=TRUE on either the user or any of their roles) bypass all permission checks. For non-admins the server looks for an exact (module, sub_module, action) grant or a broader (module, null, action) grant on any of the user's roles, then applies the scope filter (allowed_entities / allowed_warehouses / allowed_floors). Frontend can() is a UX hint only — every API call is independently authorized.

Figure 5 — Authorization (RBAC) decision tree



6. Logout, logout-all, change-password, admin reset

Figure 4 — Logout, logout-all, change-password, admin reset



7. Expected frontend payloads — endpoint by endpoint

Field types reflect modules/auth/schemas.py. min_length=1 on every string field — the frontend should validate non-empty before submit to avoid 422 round-trips.

7.1 POST /api/v1/auth/login (no auth header)

Request body

```
{
  "phone": "9876543210",                        // required, raw user input; server normalizes
  "password": "MyPass1234!",                    // required, plain text over HTTPS
  "device_info": {                              // OPTIONAL but recommended (free-form, persisted)
    "device_id":   "stable-uuid-per-install",   // stable across launches
    "device_name": "Kaushal's Pixel 8",         // user-readable label for /sessions
    "app_version": "1.1.0",
    "platform":    "android"                    // "android" | "ios" | "electron" | "web"
  }
}
```

Headers (none required). Do *not* send Authorization on login.

Validation rules: phone accepts 9876543210, 09876543210, +919876543210, 919876543210. device_info may carry extra fields — server is forward-compatible (extra=allow).

7.2 POST /api/v1/auth/refresh (no auth header)

```
{
  "refresh_token": "<current refresh JWT>"        // required
}
```

Do not send Authorization. On 200, persist *both* rotated tokens atomically.

7.3 POST /api/v1/auth/logout (Bearer required)

```
{
  "refresh_token": "<current refresh JWT>"        // required — the session to kill
}
```

Idempotent: silently 204s even for a stolen / cross-user / already-revoked token. Always clear local tokens after calling, regardless of HTTP status.

7.4 POST /api/v1/auth/logout-all (Bearer required)

Empty body. Response: { "revoked_count": N }.

7.5 GET /api/v1/auth/me (Bearer required)

No request body. No query params. Headers: Authorization: Bearer ….

7.6 POST /api/v1/auth/password/change (Bearer required)

```
{
  "old_password":     "OldPass1234",              // required
  "new_password":     "NewStrongPass5678",        // required, must satisfy §6.1 rules
  "confirm_password": "NewStrongPass5678"         // required, must == new_password
}
```

Server validates: 12–128 chars, ≥1 letter AND ≥1 digit, must not equal/contain the user's phone (≥7-digit suffix), not in common-passwords blocklist. Pre-validate the first three rules client-side for snappy UX; let the server return 400 weak_password with details.rules for the blocklist check.

7.7 GET /api/v1/auth/sessions (Bearer required)

No body. Returns the user's live refresh sessions with is_current on the active one.

7.8 DELETE /api/v1/auth/sessions/{token_id} (Bearer required)

Path param: token_id from /sessions. No body. 204 on success, 404 session_not_found if it doesn't exist *or* belongs to another user (no leak). Revoking your own current session signs you out.

8. Admin-only endpoint payloads *(Bearer + is_admin)*

8.1 POST /api/v1/auth/users — create user

```
{
  "phone":              "9876543211",            // required, accepts the same 4 formats
  "password":           "TempPass1234",          // required, same policy as /password/change
  "full_name":          "Ramesh Kumar",          // required
  "role_id":            4,                       // required, must exist in auth_role
  "email":              "ramesh@candorfoods.in",  // optional, nullable
  "entity":             "cfpl",                  // optional: "cfpl" | "cdpl" | null
  "allowed_warehouses": ["W202"]                 // optional, array of warehouse codes
}
```

409 on duplicate phone (constraint phone). 400 weak_password with details.rules if the temp password fails policy.

8.2 PUT /api/v1/auth/users/{user_id} — partial edit

```
{                                                 // every key OPTIONAL — only send what changes
  "full_name":          "Ramesh K.",
  "email":              "ramesh.k@candorfoods.in",
  "role_id":            2,
  "entity":             "cfpl",
  "is_active":          false,                   // soft-delete via DELETE is preferred
  "allowed_warehouses": ["W202","W301"]
}
```

Server allowlists fields: full_name, email, role_id, entity, is_active, allowed_warehouses. Anything else is silently dropped — sending password_encrypted, is_admin, etc. is a no-op.

8.3 DELETE /api/v1/auth/users/{user_id} — deactivate

No body. Sets is_active=FALSE and revokes all the target's live refresh tokens with revoke_reason='admin_revoked'.

8.4 POST /api/v1/auth/users/{user_id}/reset-password

```
{
  "new_password": "TempStrongPass1234"           // required, 1–256 chars, same policy as /password/change
}
```

Forces must_change_password=TRUE on the target, clears any lockout, and revokes all the target's live refresh tokens. Response includes { user_id, message, revoked_count, temp_password_set: true }.

8.5 POST /api/v1/auth/roles — create role

```
{
  "role_name":   "qc_lead",                      // required, unique
  "description": "QC team lead",                 // optional, default ""
  "is_admin":    false                           // optional, default false
}
```

8.6 PUT /api/v1/auth/roles/{role_id}/permissions — replace all

```
{
  "permission_ids":     [1, 2, 5, 10, 24, 25],   // required; replaces every existing mapping
  "allowed_entities":   ["cfpl"],                // optional, null = all entities
  "allowed_warehouses": ["W202"],                // optional, null = all warehouses
  "allowed_floors":     ["1st Floor"]            // optional, null = all floors
}
```

8.7 POST /api/v1/auth/permissions/create

```
{
  "module":         "production",                // required
  "sub_module":     "plans",                     // optional, nullable
  "sub_sub_module": "approve",                   // optional, nullable
  "action":         "create",                    // required
  "description":    "Approve a production plan"  // optional, default ""
}
```

8.8 PUT /api/v1/auth/permissions/{permission_id}

Partial edit; allowlist = module, sub_module, sub_sub_module, action, description. Any other key is silently dropped.

8.9 POST /api/v1/auth/modules — bulk create permissions

```
{
  "module":      "quality",                      // required
  "sub_modules": ["inspections", "calibration"]  // optional; null = create module-level only
}
```

```
// Auto-creates view / create / edit / delete for each (module, sub_module).
// Returns: { module, sub_modules, permissions_created: N }
```

9. Error envelope & codes

Every non-2xx response has this shape. The X-Request-ID response header always carries the same UUID as request_id. Surface the request_id in every user-facing error toast — support uses it to grep backend logs.

```
{
  "error":      "<machine_code>",                // stable; switch on this in code
  "message":    "<human_readable>",              // safe to show to users
  "request_id": "<uuid>",                        // = response header X-Request-ID
  "timestamp":  "2026-05-07T10:23:45.123Z",      // ISO-8601 UTC
  "details":    { }                              // shape varies by error (see table)
}
```

HTTP	error code	details	Frontend behaviour
400	`weak_password`	`rules: string[]`	Render rule keys inline; highlight new-password field
400	`password_mismatch`	—	Highlight confirm field
401	`invalid_credentials`	—	Generic message; do NOT distinguish unknown-phone vs wrong-password
401	`invalid_access_token`	—	Trigger refresh-and-retry once; on failure → re-login
401	`invalid_refresh_token`	—	Clear tokens, route to login
401	`token_expired`	—	From /refresh → clear + login. From normal call → impossible (interceptor)
401	`token_reuse_detected`	—	Clear tokens, route to login, show 'Security alert: please sign in again.'
401	`invalid_old_password`	—	Highlight Old Password field
403	`account_suspended`	—	'Your account is suspended. Contact your admin.'
403	`account_disabled`	—	'Your account is disabled.'
403	`forbidden`	`module, sub_module, action`	Redirect to /forbidden; show which permission was missing
404	`session_not_found`	—	On Sessions screen: 'Session no longer exists — refreshing list'
404	`user_not_found`	—	(Admin reset-password) 'User not found'
409	`varies`	—	Show message (e.g. 'Phone number already registered')
422	`validation_error`	`errors[]`	FastAPI body validation — show first error inline
423	`account_locked`	`locked_until, failed_login_count`	Disable submit; countdown to locked_until
429	`rate_limit_exceeded`	`retry_after_seconds, limit`	Disable submit; honour Retry-After header
500	`internal_error`	—	Generic + show request_id from envelope

10. Response payload quick reference

10.1 LoginResponse / RefreshResponse

```
{
  "access_token":        "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh_token":       "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type":          "Bearer",
  "expires_in":          900,                    // access TTL, seconds
  "refresh_expires_in":  28800,                  // refresh TTL, seconds
  "must_change_password": false,                 // login response only — gates navigation
  "user": {                                      // login response only
    "user_id":   "1",
    "phone":     "+919876543210",                // normalized E.164
    "full_name": "Kaushal Patel",
    "email":     "kaushal@candorfoods.in",
    "is_admin":  true,
    "roles": [
      { "role_id": "1", "code": "admin",
        "label": "Full unrestricted access", "is_admin": true }
    ]
  }
}
```

10.2 MeResponse

```
{
  "user_id":              "1",
  "phone":                "+919876543210",
  "full_name":            "Kaushal Patel",
  "email":                "kaushal@candorfoods.in",
  "status":               "active",              // "active" | "suspended" | "disabled"
  "must_change_password": false,
  "is_admin":             true,
  "roles":                [ { "role_id": "1", "code": "admin", ... } ],
  "permissions": [                               // UX-hint only; server is authoritative
    { "module": "production", "sub_module": "plans",     "action": "view"   },
    { "module": "production", "sub_module": "job_cards", "action": "view"   }
  ],
  "entities":             ["cfpl"],              // user-level scope defaults
  "warehouses":           ["W202"],
  "floors":               [],
  "last_login_at":        "2026-05-07T10:00:00Z",
  "password_changed_at":  "2026-04-30T08:30:00Z"
}
```

10.3 SessionsResponse (one row)

```
{
  "token_id":    "f81d4fae-7dec-11d0-a765-00a0c91e6bf6",
  "token_type":  "refresh",
  "issued_at":   "2026-05-07T08:00:00Z",
  "expires_at":  "2026-05-07T16:00:00Z",
  "ip":          "203.0.113.42",
  "user_agent":  "Mozilla/5.0 ...",
  "device_info": {                               // whatever the client sent on /login
    "device_id":   "stable-uuid",
    "device_name": "Kaushal's Pixel 8",
    "platform":    "android"
  },
  "is_current":  true                            // matches current refresh jti
}
```

11. Source pointers

Concern	File
Endpoints + wire shapes	`modules/auth/router.py` · `modules/auth/schemas.py`
Login / refresh / rotation	`modules/auth/services/auth_service.py`
JWT issuance + verification	`modules/auth/services/jwt_service.py`
Permission engine	`modules/auth/services/permission_service.py`

Password rules	`modules/auth/services/password_rules.py`
Phone normalization	`modules/auth/services/phone.py`
Rate limiter	`modules/auth/services/rate_limiter.py`
RBAC middleware	`modules/auth/middleware.py`
Error envelope middleware	`core/middleware/request_context.py`
DB schema	`db/auth_schema.sql`
Token TTL / lockout settings	`app/config.py (Settings.ACCESS_TOKEN_TTL_SECONDS, …)`